

DeepSeek Coder

Deep Dive into DeepSeek — Chapter 16

Yin Yang

Foundation Books Project

The one rule of this chapter

A model output is not a repository change.
The gap between them is **client-owned work**.

- Code specialization changes the corpus, the objective, and the backbone — and each release changes the *request protocol* too.
- But a generated artifact still has to be reconstructed or applied, kept inside an allowed edit scope, parsed, and executed.
- Those last stages belong to the client, not to the checkpoint.

Goal: follow one small configuration-parser task — labeled `boolcfg` — from training mixture and FIM serialization through patch application, edit scope, and a bounded behavioral checker.

1. The code-task contract

Roadmap

1. The code-task contract
2. Coder lineage and training

Roadmap

1. The code-task contract
2. Coder lineage and training
3. FIM and repository context

Roadmap

1. The code-task contract
2. Coder lineage and training
3. FIM and repository context
4. Reading the official evaluations

1. The code-task contract
2. Coder lineage and training
3. FIM and repository context
4. Reading the official evaluations
5. Protocols, artifacts, and safe execution

Roadmap

1. The code-task contract
2. Coder lineage and training
3. FIM and repository context
4. Reading the official evaluations
5. Protocols, artifacts, and safe execution
6. What the `boolcfg` harness measured

The code-task contract

Definition (Code-task contract)

A code task is $T = (R, q, H, S)$: base repository state R , requested change q , evaluation harness H , and allowed edit scope S . A generated artifact becomes a candidate patch δ *only if it applies to R , and*

$$P_H(\delta; R, S) = 1 \iff \delta \text{ applies, } \delta \text{ respects } S, H(\delta(R)) \text{ succeeds.}$$

P_H is a conjunction, not a score. Three independent ways to be zero — and a single pass rate hides which one fired.

Three successive events, each able to fail

task
 R, q, S

Three successive events, each able to fail

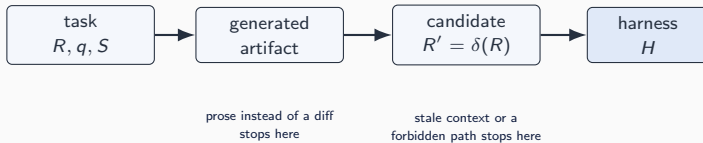


prose instead of a diff
stops here

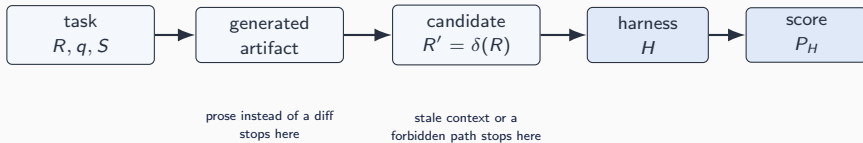
Three successive events, each able to fail



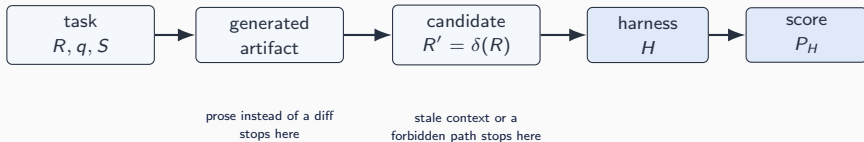
Three successive events, each able to fail



Three successive events, each able to fail

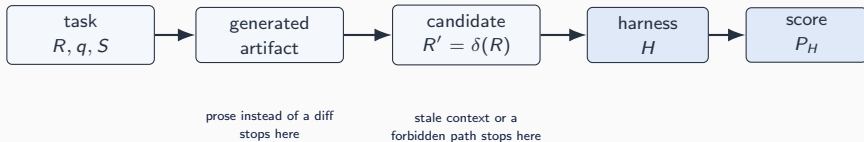


Three successive events, each able to fail



- The running `boolcfg` case: the parser compares raw input with lowercase Boolean literals *before* trimming or normalizing it.
- R is the base commit, q asks that mixed-case and padded values work, H is the visible test command, and S permits parser edits but not test edits.

Three successive events, each able to fail



- The running `boolcfg` case: the parser compares raw input with lowercase Boolean literals *before* trimming or normalizing it.
- R is the base commit, q asks that mixed-case and padded values work, H is the visible test command, and S permits parser edits but not test edits.

A cleanly applied edit to the test file violates S even when the weakened suite exits successfully.

Coder lineage and training

One family, two very different engines

Release object	Source-backed identity	Use consequence
Original Coder Base	1.3B, 6.7B, 33B dense; GQA in the 33B	raw continuation and sentinel FIM surfaces
Original Coder Instruct	instruction-tuned at a named scale	conversation in, code possibly wrapped in prose
Original Coder 5.7B MQA Base	Base-only MQA exception in the release	no matching reported Instruct checkpoint
Coder-V2 Lite	16B total, 2.4B active; FIM training reported	V2-derived MoE with Base, Instruct, and infilling use
Coder-V2 full	236B total, 21B active; next-token pretraining reported	largest sparse capacity, without the Lite FIM objective

[REPORTED: DeepSeek-Coder and Coder-V2 reports and repositories]

“Coder-V2 supports FIM” is too broad unless it names the variant and the invocation surface.

Two training programs, not one architecture swap

Dimension	Original Coder	Coder-V2
Starting point	dense code-family checkpoints	intermediate DeepSeek-V2 checkpoint at 4.2T reported tokens
Added training	2T tokens: 87% code, 10% English code-related, 3% Chinese	6T added tokens: 60% code, 10% mathematics, 30% natural language
Repository signal	dependency-based file ordering, repo-level near-deduplication, quality filtering	continued code training; language record expanded to 338
Objective	next-token mixed with a selected 50% PSM FIM policy	FIM for the 16B Lite line; next-token for the reported 236B run
Post-training	Base and instruction-tuned releases	SFT, compiler/test-derived reward modeling, GRPO

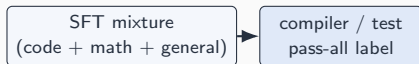
[REPORTED: DeepSeek-Coder and Coder-V2 technical reports]

Six coordinates move at once: backbone, mixture, FIM exposure, context budget, post-training, and evaluation protocol.

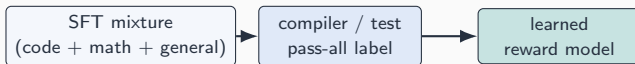
Compiler results do not update the policy directly

SFT mixture
(code + math + general)

Compiler results do not update the policy directly



Compiler results do not update the policy directly



Compiler results do not update the policy directly



Compiler results do not update the policy directly



- The binary pass-all-tests label is useful but noisy: limited coverage can miss incorrect behavior.
- So the reported path trains a reward model on compiler-provided preference data and feeds *reward-model scores* into GRPO.
- The reported in-house LeetCode comparison prefers the learned signal over the raw compiler signal. [REPORTED: DeepSeek-Coder-V2 technical report]

Compiler results do not update the policy directly



- The binary pass-all-tests label is useful but noisy: limited coverage can miss incorrect behavior.
- So the reported path trains a reward model on compiler-provided preference data and feeds *reward-model scores* into GRPO.
- The reported in-house LeetCode comparison prefers the learned signal over the raw compiler signal. [REPORTED: DeepSeek-Coder-V2 technical report]

Training feedback shapes the policy. It does not validate a later patch against R , S , and H .

FIM and repository context

PSM is a presentation order, not a program order

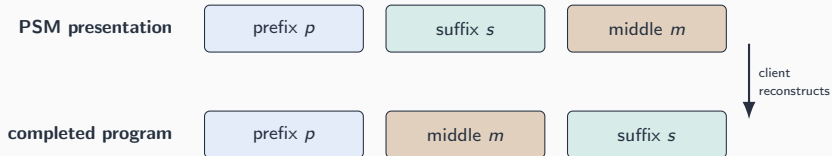
PSM presentation

prefix p

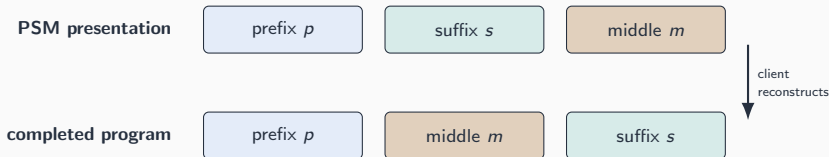
suffix s

middle m

PSM is a presentation order, not a program order



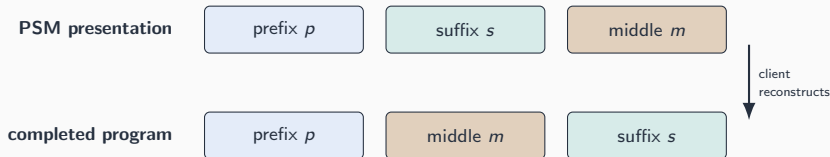
PSM is a presentation order, not a program order



Invariant (FIM reconstruction boundary)

The client preserves p and s , inserts only the generated middle m , and validates $p||m||s$. Local sentinel prompts and hosted prefix/suffix fields are two serializations of one operation.

PSM is a presentation order, not a program order



Invariant (FIM reconstruction boundary)

The client preserves p and s , inserts only the generated middle m , and validates $p||m||s$. Local sentinel prompts and hosted prefix/suffix fields are two serializations of one operation.

Acceptance is a product: $P_{\text{syntax}} P_{\text{scope}} P_{\text{tests}}(p||m||s) = 1$.

Context packing is a compression problem

$C(R, q, B) \rightarrow x$ selects, orders, and possibly truncates repository evidence within budget B , and

$$B_{\text{source}} + B_{\text{control}} + B_{\text{output}} + B_{\text{margin}} \leq L_{\text{ctx}}.$$

Filling L_{ctx} with source files can make an otherwise relevant patch impossible to finish.

Prompt component	Toy tokens
Issue text	120
Relevant tests	260
Target function	220
Exception definition	90
Unrelated formatter	500
Budget 1,200. The first four cost 690 and leave 510.	

Every retained object should lie on an evidence path to the issue, scope, dependency, test, public contract, or build condition.

From concept to code: repository context is a prompt region

The official repository-completion example serializes several files into one prompt and stops inside the target function.

```
from DeepSeek-Coder/README.md
```

```
input_text = """#utils.py
```

```
...
```

```
def load_data():
```

```
...
```

```
# main.py
```

```
from utils import load_data, evaluate_predictions
```

```
from model import IrisClassifier as Classifier
```

```
def main():
```

```
    # Model training and evaluation
```

```
"""
```

From concept to code: repository context is a prompt region

The official repository-completion example serializes several files into one prompt and stops inside the target function.

```
from DeepSeek-Coder/README.md

input_text = """#utils.py
...
def load_data():
...
# main.py
from utils import load_data, evaluate_predictions
from model import IrisClassifier as Classifier

def main():
    # Model training and evaluation
    """
```

DeepSeek-Coder-6.7B Base completes `main()` by *calling* the imported helpers and classifier instead of redefining them. Selection, order, and budget are still the client's decisions.

[INSPECTED: DeepSeek-Coder README.md]

Reading the official evaluations

One name, several capabilities

Release and surface	Reported	Technical pattern
Coder-V2 Instruct, HumanEval	90.2	strong specification-to-code
Coder-V2 Instruct, MBPP+	76.2	a second distribution holds up
Coder-V2 Instruct, LiveCodeBench	43.4	fresh contests stay harder
Coder-V2 Lite Base, RepoBench	Py 38.9; Java 43.3	repository-conditioned completion varies by language
Coder-V2 Lite Base, HumanEval FIM	86.4	Lite Base retains insertion
Coder-V2 Instruct, repair/editing	Defects4J 21.0; SWE-Bench 12.7; Aider 73.7	issue-to-patch and tool-mediated editing are different protocols

[REPORTED: DeepSeek-Coder-V2 report and repository]

12.7 next to 73.7 is not a contradiction. It is the price of the extra operations: base-revision control, patch application, and hidden tests.

Protocols, artifacts, and safe execution

Three protocols, three client reconstructions

Coder surface	Model input and output	Client reconstruction
Base completion	raw source prefix; generated continuation	append under the chosen stop rule

Three protocols, three client reconstructions

Coder surface	Model input and output	Client reconstruction
Base completion	raw source prefix; generated continuation	append under the chosen stop rule
Open-weight FIM	prefix and suffix serialized with tokenizer sentinels; generated middle	insert the middle between an unchanged p and s

Three protocols, three client reconstructions

Coder surface	Model input and output	Client reconstruction
Base completion	raw source prefix; generated continuation	append under the chosen stop rule
Open-weight FIM	prefix and suffix serialized with tokenizer sentinels; generated middle	insert the middle between an unchanged p and s
Instruct chat	conversation with request and selected repository evidence; instruction response	extract the requested code, replacement, or diff

Three protocols, three client reconstructions

Coder surface	Model input and output	Client reconstruction
Base completion	raw source prefix; generated continuation	append under the chosen stop rule
Open-weight FIM	prefix and suffix serialized with tokenizer sentinels; generated middle	insert the middle between an unchanged p and s
Instruct chat	conversation with request and selected repository evidence; instruction response	extract the requested code, replacement, or diff

Shortest useful rejection path: **(1)** reconstruct or apply, **(2)** reject edits outside the allowed paths, **(3)** parse or compile, **(4)** run the fixed behavioral checker.

Three protocols, three client reconstructions

Coder surface	Model input and output	Client reconstruction
Base completion	raw source prefix; generated continuation	append under the chosen stop rule
Open-weight FIM	prefix and suffix serialized with tokenizer sentinels; generated middle	insert the middle between an unchanged p and s
Instruct chat	conversation with request and selected repository evidence; instruction response	extract the requested code, replacement, or diff

Shortest useful rejection path: **(1)** reconstruct or apply, **(2)** reject edits outside the allowed paths, **(3)** parse or compile, **(4)** run the fixed behavioral checker.

The first failed stage separates a request-protocol failure from a program failure.

From concept to code: serialization is protocol-critical

The Coder-V2 example fixes the tokenizer, the repository implementation, the dtype, and the exact chat serialization.

```
from DeepSeek-Coder-V2/README.md

tokenizer = AutoTokenizer.from_pretrained("deepseek-ai/DeepSeek-Coder-V2-Lite-Instruct", trust_remote_code=
    True)
model = AutoModelForCausalLM.from_pretrained(..., trust_remote_code=True, torch_dtype=torch.bfloat16).cuda
    ()
inputs = tokenizer.apply_chat_template(messages, add_generation_prompt=True, return_tensors="pt").to(model.
    device)
```

From concept to code: serialization is protocol-critical

The Coder-V2 example fixes the tokenizer, the repository implementation, the dtype, and the exact chat serialization.

```
from DeepSeek-Coder-V2/README.md

tokenizer = AutoTokenizer.from_pretrained("deepseek-ai/DeepSeek-Coder-V2-Lite-Instruct", trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(..., trust_remote_code=True, torch_dtype=torch.bfloat16).cuda()
inputs = tokenizer.apply_chat_template(messages, add_generation_prompt=True, return_tensors="pt").to(model.device)
```

The published template ends the prompt with `Assistant:` and no trailing space; the repository warns that adding a space can produce Chinese, garbled, or repetitive output on the 16B-Lite model. Loading, checkpoint type, and exact serialization are all protocol. [INSPECTED:

DeepSeek-Coder-V2 README.md]

Generated code stays untrusted until a checker accepts it

Contract (Code-serving safety)

Generated code must not receive broader filesystem, network, secret, or process privileges than the task requires. Model output remains untrusted until the sandbox and executable checker accept it.

- A static scan can reject recognizable unsafe calls; it cannot prove runtime behavior.
- A read-only filesystem limits persistent writes, not computation or disclosure. A timeout bounds elapsed work, not every resource spike.
- A no-network, capability-dropped container with no secrets mounted sharply reduces available effects — and still trusts the kernel and the confinement backend.

Name the controls and the residual risk. Do not summarize either as “safe execution.”

**What the boolcfg harness
measured**

A zero aggregate that still diagnoses something

Interface	Missing patch	Unapplied diff	Forbidden path	Syntax error	Failed tests
General chat	4	5	1	0	0

A zero aggregate that still diagnoses something

Interface	Missing patch	Unapplied diff	Forbidden path	Syntax error	Failed tests
General chat	4	5	1	0	0
Coder chat	2	8	0	0	0

A zero aggregate that still diagnoses something

Interface	Missing patch	Unapplied diff	Forbidden path	Syntax error	Failed tests
General chat	4	5	1	0	0
Coder chat	2	8	0	0	0
FIM-labeled path	0	0	0	1	9

A zero aggregate that still diagnoses something

Interface	Missing patch	Unapplied diff	Forbidden path	Syntax error	Failed tests
General chat	4	5	1	0	0
Coder chat	2	8	0	0	0
FIM-labeled path	0	0	0	1	9
Repository context	0	9	1	0	0

A zero aggregate that still diagnoses something

Interface	Missing patch	Unapplied diff	Forbidden path	Syntax error	Failed tests
General chat	4	5	1	0	0
Coder chat	2	8	0	0	0
FIM-labeled path	0	0	0	1	9
Repository context	0	9	1	0	0

First rejection by interface, revision-pinned isolated July 14 run, ten candidates each. The full 120-response strict aggregate contains no accepted patch. [OBSERVED: boolcfg patch-harness campaigns]

Chat and repository-context candidates mostly died before syntax checking; the FIM-labeled candidates reached the tests and failed there. That is a stage-transition diagnosis, not a ranking of coding ability.

Concept-to-code map for the chapter

- **dense Coder lineage, mixture, and release roles** → `DeepSeek-Coder/README.md`
- **repository-level completion prompt** → `DeepSeek-Coder/README.md`
- **benchmark invocation for one suite** → `DeepSeek-Coder/Evaluation/HumanEval/README.md`
- **loading, FIM serialization, chat template** → `DeepSeek-Coder-V2/README.md`
- **the 338-language record** → `DeepSeek-Coder-V2/supported_langs.txt`
- **four request interfaces, FIM sentinel prompt** →
`experiments/volume_i_coder/deepseek_boolcfg_sampler.py`
- **strict patch harness and its self-test** → `experiments/volume_i_coder/boolcfg_patch_harness.py`

Identify the checkpoint, pin the protocol, own the reconstruction, gate on the checker.

Next: the multimodal chapter carries the same gap between generated content and a validated artifact into images and generation.